EXPERIMENT 01 Bernstein–Vazirani Algorithm

Aim

To recover a hidden binary string using a quantum oracle.

Objectives

1. Construct an oracle encoding a secret bit string.
2. Apply Hadamard transformations and phase kickback.
3. Measure the input register to recover the string.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook.

Theory

The Bernstein–Vazirani algorithm identifies an unknown n-bit string s when an oracle computes the parity of its bitwise product with an input x. The input register is prepared in equal superposition and an auxiliary qubit in the $|-\rangle$ state. Controlled-NOT operations encode the oracle; phase kickback changes the input amplitudes according to s. A final Hadamard layer makes the hidden string measurable in one ideal oracle query, rather than n classical deterministic queries.

Key Equation(s) $f_s(x) = s \cdot x \bmod 2$; $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f_s(x)\rangle$; $H^{\otimes n} U_f$ $H^{\otimes n}|0^n\rangle|-\rangle \rightarrow |s\rangle|-\rangle$.

Algorithm / Steps of Execution

1. Define a binary secret string and its length n.
2. Create n input qubits, one auxiliary qubit, and n classical bits.
3. Initialize the auxiliary qubit to $|1\rangle$ and apply Hadamard to all qubits.
4. Build the oracle using CNOT for each 1-bit in the secret string.
5. Apply Hadamard gates to input qubits and measure them.
6. Run the simulator, plot counts, and compare the output with s.

Applications

1. Quantum oracle demonstrations
2. Query-complexity education
3. Phase-kickback experiments

Notebook: EXP-01.ipynb

Aim

EXPERIMENT 02

Simon's Algorithm

To investigate a hidden XOR period using a quantum oracle.

Objectives

1. Construct a two-register quantum circuit.
2. Measure outputs satisfying the Simon orthogonality condition.
3. Interpret measurement equations to recover the hidden period.

Requirements

Python, Qiskit, Qiskit Aer, Matplotlib, Jupyter Notebook.

Theory

Simon's problem assumes a function with a hidden nonzero string s such that f(x)=f(y) precisely when y=x or y=x $\oplus$ s. Hadamard gates prepare a superposition, an oracle computes the function, and a second Hadamard layer produces measurement strings orthogonal to s over binary arithmetic. Repeating the circuit supplies linear equations from which s can be determined. The supplied practical demonstrates an oracle construction and sampled results; a complete implementation should additionally solve the binary equations and verify the promised two-to-one property.

Key Equation(s) f(x)=f(x $\oplus$s); $y \cdot s = 0 \pmod 2$; $y_1 \cdot s = \ldots = y_k \cdot s = 0 \pmod 2$.

Algorithm / Steps of Execution

1. Choose a nonzero hidden XOR string s.
2. Allocate input and output quantum registers plus classical bits.
3. Apply Hadamard gates to the input register.
4. Construct and apply the function oracle with CNOT gates.
5. Apply Hadamard gates again and measure the input register.
6. Collect repeated outcomes and test the parity equations y·s=0.

Applications

1. Hidden-period demonstrations

Aim

2. Quantum query algorithms
3. Binary linear algebra over GF(2)

Notebook: EXP-02.ipynb

EXPERIMENT 03 Grover Search Algorithm — Two Qubits

To amplify the probability of a marked state in a four-element search space.

Objectives

1. Prepare a uniform two-qubit superposition.
2. Apply a phase oracle and diffusion operator.
3. Measure the marked state after amplitude amplification.

Requirements

Python, Qiskit, Qiskit Aer, Matplotlib, Jupyter Notebook.

Theory

Grover's algorithm searches an unstructured set by alternating an oracle that changes the sign of a marked state with a diffusion operation that reflects amplitudes about their average. For two qubits there are four basis states, and one ideal Grover iteration can amplify a single marked state to unit probability. The uploaded circuit marks $|11\rangle$ using a controlled-Z operation, performs diffusion, and measures the resulting state.

Key Equation(s)

$N = 2^n$; $|u\rangle = (1/\sqrt{N})\Sigma_x|x\rangle$; $G = (2|u\rangle\langle u| - I)O$; $r \approx (\pi/4)\sqrt{(N/M)}$.

Algorithm / Steps of Execution

1. Create a two-qubit quantum circuit with two classical bits.
2. Apply Hadamard gates to create a uniform superposition.
3. Mark $|11\rangle$ with a controlled-Z phase oracle.
4. Apply the H–X–CZ–X–H diffusion sequence.
5. Measure both qubits and simulate using fixed shots.
6. Inspect the histogram and confirm the marked outcome dominates.

Aim

Applications

1. Unstructured search examples
2. Amplitude-amplification studies
3. Quantum algorithm teaching

Notebook: EXP-03.ipynb

EXPERIMENT 04 Grover Search Algorithm — Three Qubits

To implement amplitude amplification for a marked state among eight candidates.

Objectives

1. Prepare a three-qubit uniform superposition.
2. Construct a multi-controlled phase oracle.
3. Repeat oracle and diffusion stages and analyze results.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook.

Theory

For three qubits the search space contains eight computational basis states. A phase oracle marks a selected state, here $|111\rangle$, by reversing its amplitude sign. The diffusion operator increases the marked state's probability through interference. The uploaded implementation uses a CCX gate surrounded by Hadamard gates to realize a controlled phase operation and repeats the Grover iteration twice. The ideal success probability depends on the number of iterations and the number of marked states.

Key Equation(s)

$N = 8$, $M = 1$; $\theta = \arcsin(\sqrt{(M/N)})$; $P(\text{success after r iterations}) = \sin^2((2r+1)\theta)$.

Algorithm / Steps of Execution

1. Create a three-qubit circuit and classical measurement register.
2. Apply Hadamard to all three qubits.
3. Implement the $|111\rangle$ phase oracle using H–CCX–H.
4. Implement diffusion using H, X, multi-controlled phase, X, H.

Aim

5. Repeat the oracle–diffusion cycle as in the supplied practical.
6. Measure, simulate, and compare marked-state frequency with theory.

Applications

1. Multi-qubit search circuits
2. Iteration-count optimization
3. Oracle and diffusion design

Notebook: EXP-04.ipynb

EXPERIMENT 05 Quantum Fourier Transform and Inverse QFT

To implement QFT and verify recovery of a prepared computational basis state.

Objectives

1. Construct a three-qubit QFT circuit.
2. Apply the inverse transform using reversed gate order.
3. Check whether measurement recovers the input state.

Requirements

Python, Qiskit, Qiskit Aer, NumPy, Jupyter Notebook.

Theory

The Quantum Fourier Transform maps computational basis states to phase-encoded superpositions. Its circuit uses Hadamard gates, controlled phase rotations, and swaps to manage bit order. The inverse QFT reverses the operation and uses conjugate phase rotations. In the supplied practical, a basis state is prepared, QFT is applied, and then inverse QFT is performed; the final measurement should return the original basis state in an ideal simulation.

Key Equation(s)

$QFT_N|x\rangle = (1/\sqrt{N})\Sigma_{k=0}^{N-1} \exp(2\pi i x k/N)|k\rangle$; $QFT\dagger\ QFT = I$; $N=2^n$.

Algorithm / Steps of Execution

1. Create a three-qubit circuit and prepare $|001\rangle$ as in the source.
2. Apply Hadamard and controlled-phase rotations for QFT.

Aim

3. Apply a swap to correct output qubit order.
4. Apply inverse swaps and negative-angle controlled-phase gates.
5. Complete inverse QFT and measure all qubits.
6. Simulate and verify recovery of the prepared basis state.

Applications

1. Quantum phase estimation
2. Period-finding subroutines
3. Quantum signal and phase analysis

Notebook: EXP-05.ipynb

EXPERIMENT 06 Factoring 15 Demonstration

To examine the period-finding stages used in a small-number Shor demonstration.

Objectives

1. Select a coprime base and target integer.
2. Build the circuit stages for modular periodicity and QFT.
3. Interpret measurement outcomes as possible period information.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook.

Theory

Shor's factoring algorithm reduces integer factorization to order finding: for a number N and an integer a coprime to N, determine the smallest positive r such that $a^r \equiv 1 \pmod{N}$. Quantum phase estimation and an inverse QFT can reveal information about r; classical post-processing may then yield factors using greatest common divisors. The uploaded practical is a simplified factoring-15 demonstration. Its circuit and outcomes should not be presented as a complete general modular-exponentiation implementation without additional verification.

Key Equation(s) $a^r \equiv 1 \pmod{N}$; gcd(a,N)=1; factors may be gcd(a^(r/2) ± 1,N) when r is even.

Aim

Algorithm / Steps of Execution

1. Choose N=15 and a base a coprime to 15.
2. Allocate the counting and work registers in a quantum circuit. 3. Prepare counting-register superposition with Hadamard gates.
4. Implement the supplied controlled operations representing periodicity.
5. Apply inverse QFT and measure the counting register.
6. Interpret measured phases, estimate r, and check candidate factors.

Applications

1. Quantum factoring education
2. Order-finding demonstrations
3. Quantum cryptanalysis fundamentals

Notebook: EXP-06.ipynb

EXPERIMENT 07 Quantum Support Vector Machine with Quantum Kernel

To train a classifier using a quantum-inspired kernel estimation workflow.

Objectives

1. Prepare a small labeled dataset.
2. Estimate a kernel matrix using quantum state overlap.
3. Train and evaluate an SVM classifier.

Requirements

Python, NumPy, scikit-learn, Qiskit, Jupyter Notebook.

Theory

A quantum kernel method maps classical feature vectors into quantum states through a feature map. Pairwise similarity is computed from the squared overlap of encoded states, producing a kernel matrix suitable for a support vector machine. The uploaded experiment constructs training and test data, defines a quantum feature-map circuit and uses a simulated kernel with an SVM. Accuracy on a small example is a demonstration only; it does not establish quantum advantage or generalization.

Aim

Key Equation(s)

$K(x,z) = |\varphi(x)|\varphi(z)|^2$; $f(x) = \Sigma_i \alpha_i y_i K(x_i,x) + b$.

Algorithm / Steps of Execution

1. Import NumPy, scikit-learn and Qiskit libraries.
2. Define and split the small labeled dataset.
3. Create a parameterized quantum feature map for each sample.
4. Estimate pairwise overlaps to form training and test kernel matrices.
5. Fit an SVM using the precomputed kernel.
6. Predict test labels and report accuracy and dataset size.

Applications

1. Quantum machine learning prototypes
2. Kernel-method comparisons
3. Hybrid classical–quantum classification

Notebook: EXP-07.ipynb

EXPERIMENT 08

Quantum Key Distribution BB84-Style Simulation

To simulate random basis selection and compare sender and receiver bits.

Objectives

1. Generate random sender bits and encoding bases.
2. Simulate receiver measurement basis choices.
3. Identify matching-basis positions for key sifting.

Requirements

Python, NumPy, Qiskit, Jupyter Notebook.

Aim

Theory

The BB84 protocol encodes random classical bits into quantum states chosen from two incompatible bases. A receiver independently chooses measurement bases; sender and receiver later announce their bases over an authenticated classical channel and retain only positions where the bases match. The uploaded notebook generates random bit and basis arrays and performs a simplified simulation of this process. A complete secure protocol also requires eavesdropping estimation, error correction, privacy amplification, and authentication.

Key Equation(s)

Z basis: {|0 ,|1 }; X basis: {|+ ,|− }; |± =(|0 ±|1)/√2; keep i when basis_A(i)=basis_B(i).

Algorithm / Steps of Execution

1. Generate random sender bits and random sender bases.
2. Encode each bit using the selected Z or X basis.
3. Generate random measurement bases for the receiver.
4. Simulate measurements and record receiver outcomes.
5. Publicly compare bases and retain only matching-basis positions.
6. Compare sifted keys and calculate the observed error rate.

Applications

1. Quantum communication education
2. Key-sifting demonstrations
3. Eavesdropping detection foundations

Notebook: EXP-08.ipynb